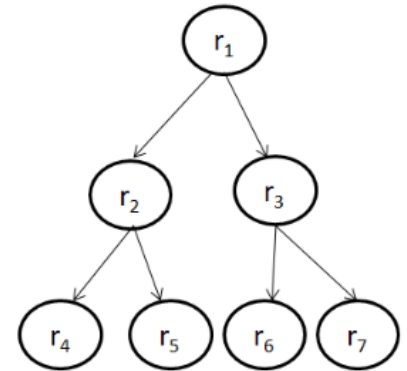


Algorithms and data structures II

Dr Chris Roadknight

Reminder-The power of logs

- The impact of turning an $O(n)$ problem to an $O(\log(n))$ is often lost when visual (~ 10 element) examples are used



n	log(n)
10	3.321928
100	6.643856
1,000	9.965784
10,000	13.28771
100,000	16.60964
1,000,000	19.93157

Cost vs benefit of structuring your data.

For instance, when is (A) a better solution to (B)

	Solution A	Solution B
Build	$O(1)$	$O(\log n)$
Insert	$O(1)$	$O(\log n)$
Delete	$O(1)$	$O(\log n)$
Search	$O(n)$	$O(\log n)$

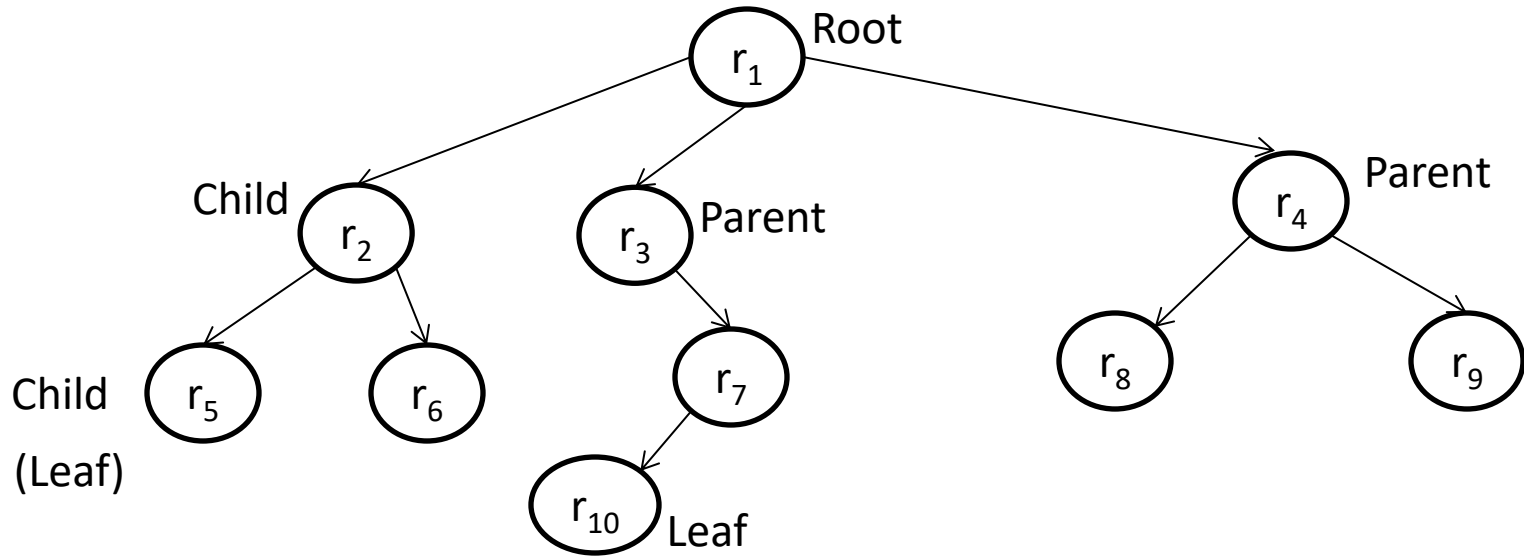
Divide and Conquer

- Higher or lower, think of a number between 1 and 63
- First try, 1,2,3,4,5,6,7.....
 - $O(n)$
- Second try, ..higher or lower?
 - $O(\log n)$
- ...but...requires **feedback**..which requires **order**

Tree Review: preliminary definitions

- The successor nodes of a node are called its **children**
- The predecessor node of a node is called its **parent**
- The "beginning" node is called the **root** (has no parent)
- A node without children is called a **leaf**

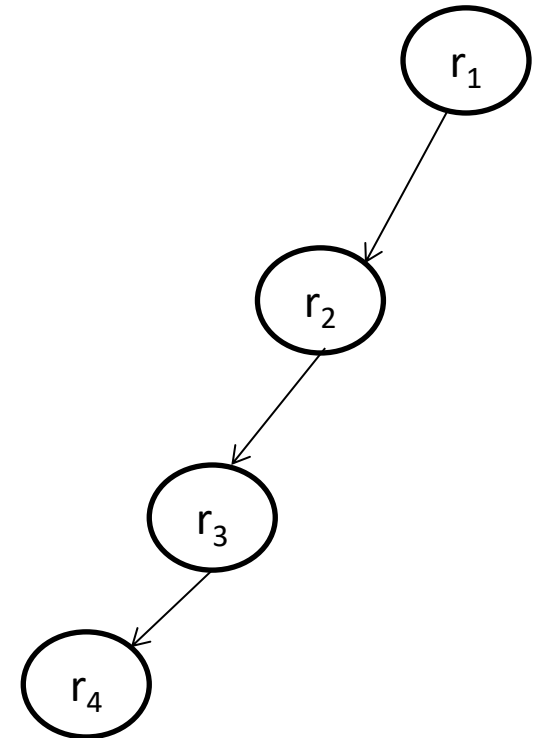
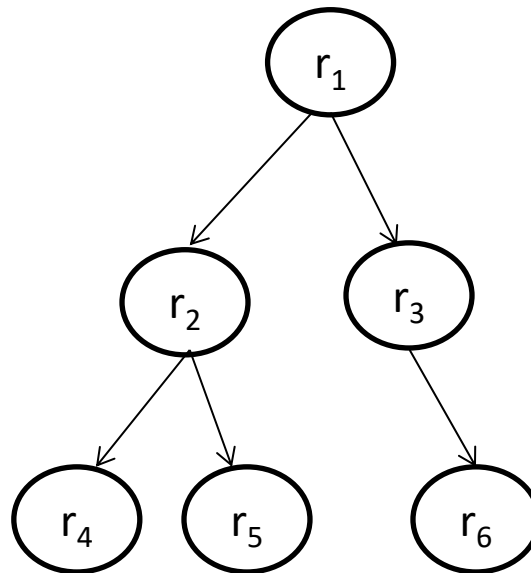
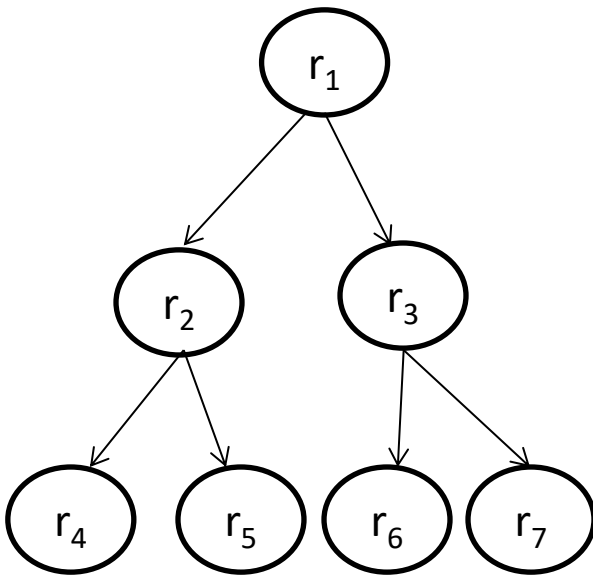
Tree: preliminary definitions



- The length of a path is the number of edges on the path, e.g., a path $\{r_3, r_7, r_{10}\}$ having length 2 i.e., one less than the number of nodes in the path
- The depth of a node r_i is the length of the unique path from the root to r_i . Thus the root is at depth 0 and depth of r_6 is 2
- The height of a node r_i is the length of the longest path from r_i to a leaf. E.g., height of r_3 is 2. (What is the height of a leaf node?)
- The height of a tree is the height of its root

Binary tree

- A binary tree T is a tree in which each node has at most two child nodes. It has a root. The remaining elements (if any) are partitioned into two binary trees, called left and right subtrees of T

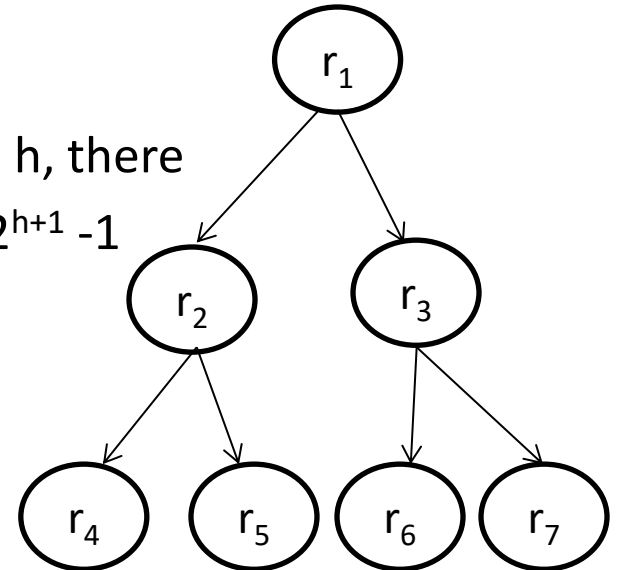


Perfect binary tree

- A full binary tree is a tree in which every internal node has two children. A perfect binary tree is a full binary tree in which all leaves are at the same level
- In a perfect (or complete) binary tree with height h , there
- are 2^h leaf nodes and total number of nodes $n = 2^{h+1} - 1$

$$2^{h+1} = n + 1$$

$$h + 1 = \log_2 (n+1)$$

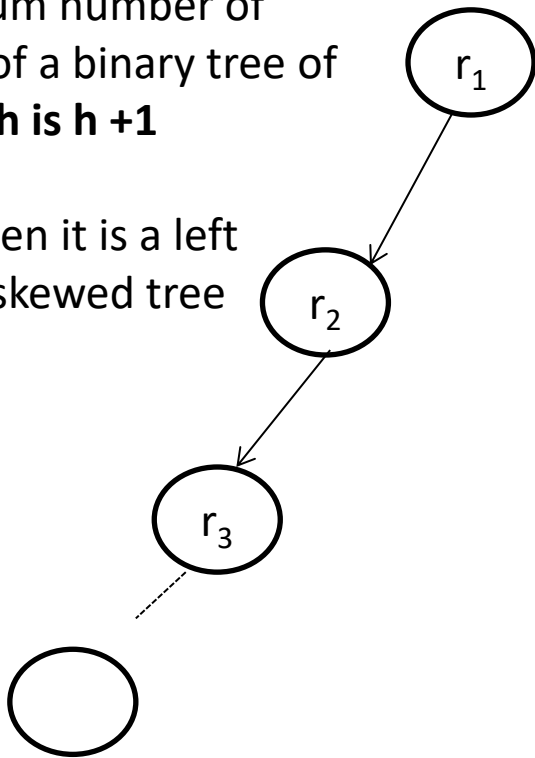


We can say height of a perfect binary tree of n nodes is $O(\log(n))$

Min and Max number of nodes in a Binary tree

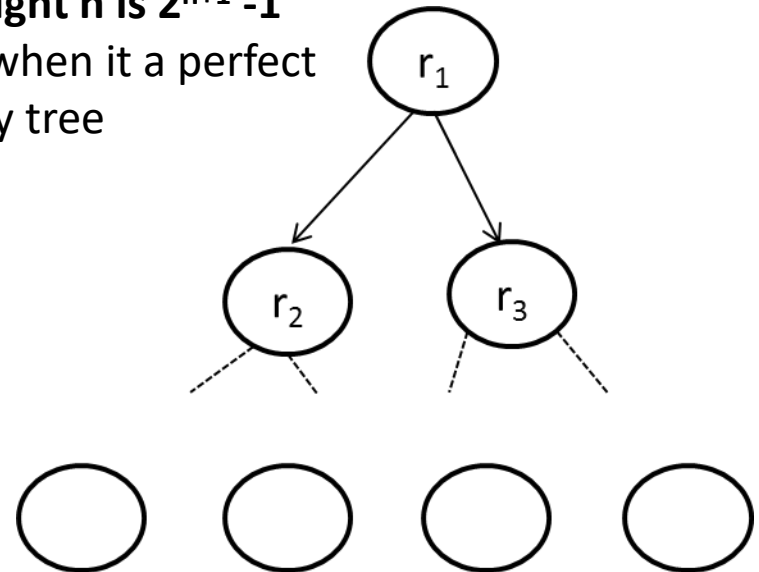
Minimum number of nodes of a binary tree of **height h is $h + 1$**

I.e., when it is a left (right) skewed tree

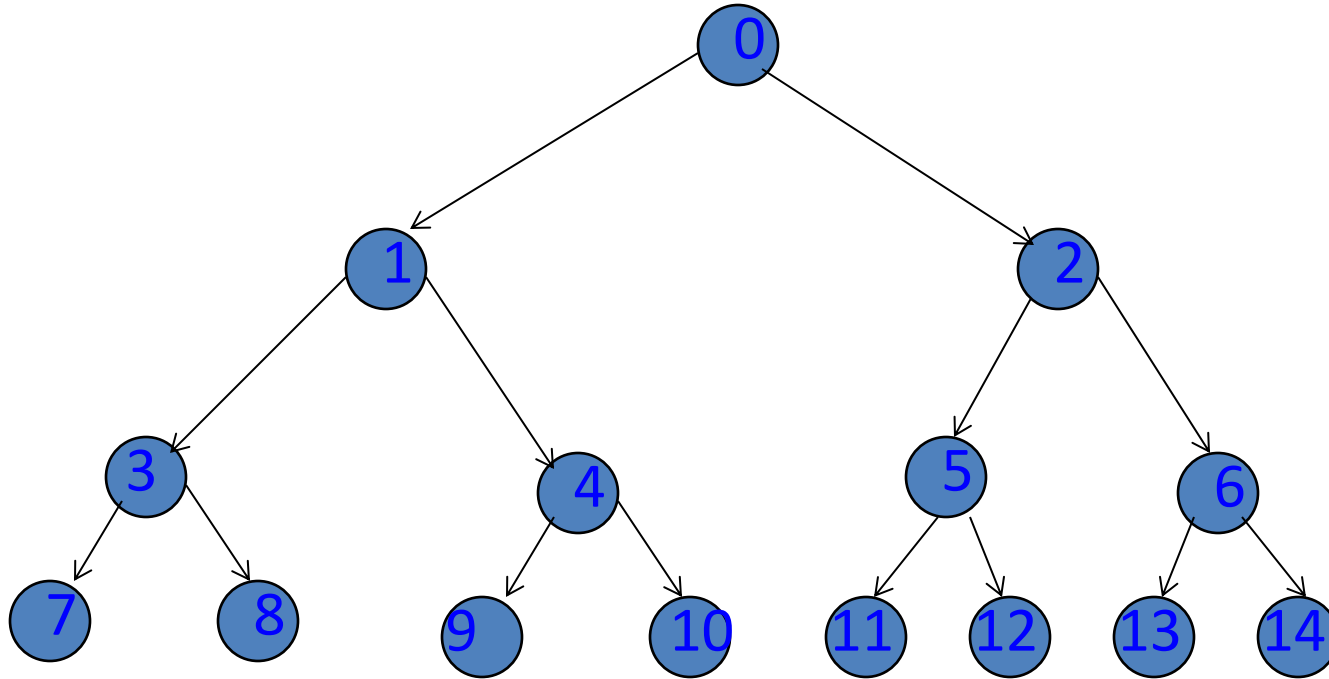


Maximum number of nodes of a binary tree of **height h is $2^{h+1} - 1$**

I.e., when it is a perfect binary tree



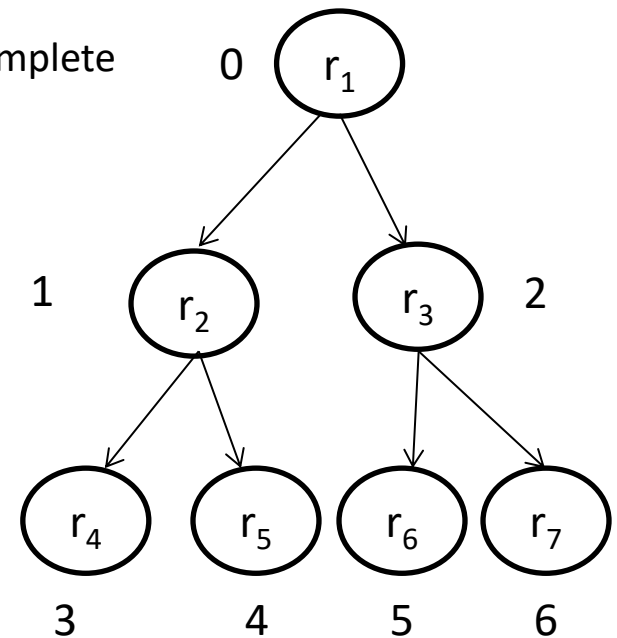
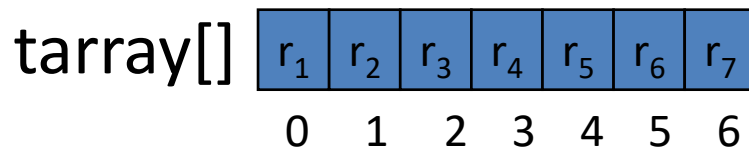
Node numbering



- Parent of node i ($i \neq 0$) is node $(i-1) / 2$ (integer division)
- Left child of node i is $2i+1$ if $2i+1 < n$, where n is the number of nodes
- Right child of node i is $2i+2$ if $2i+2 < n$, where n is the number of nodes

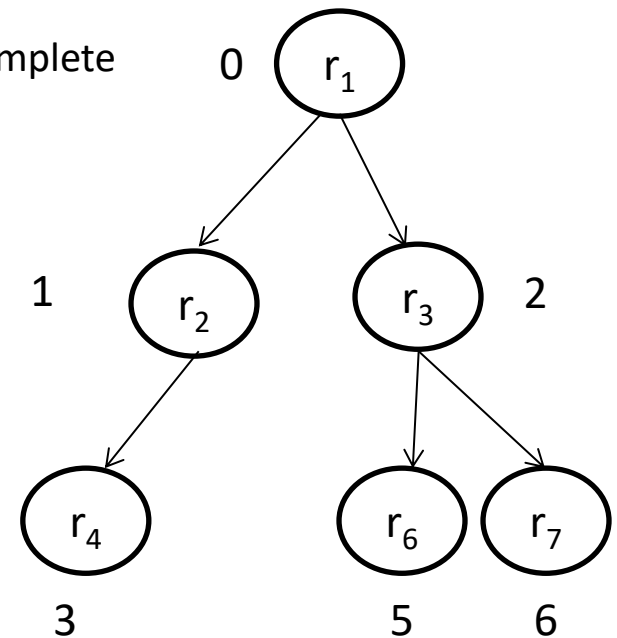
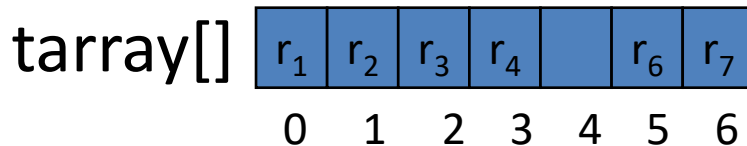
Implementation of binary tree

- Array implementation
 - number the nodes using the numbering scheme for a complete binary tree
 - node number i is stored at `tarray[i]`



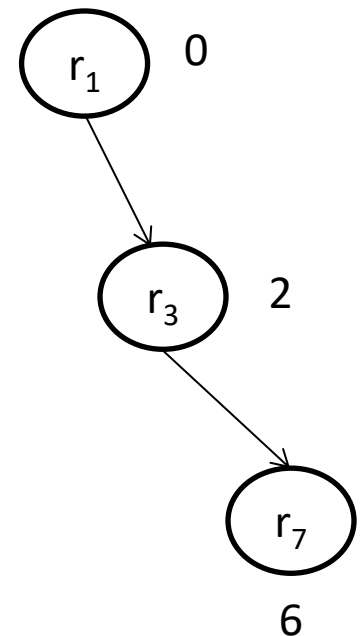
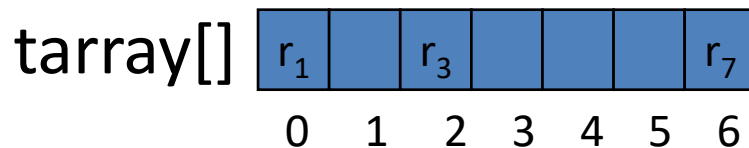
Implementation of binary tree contd.

- Array implementation
 - number the nodes using the numbering scheme for a complete binary tree
 - node number i is stored at `tarray[i]`



Implementation of binary tree contd.

- Array implementation
 - number the nodes using the numbering scheme for a complete binary tree
 - node number i is stored at $tarray[i]$

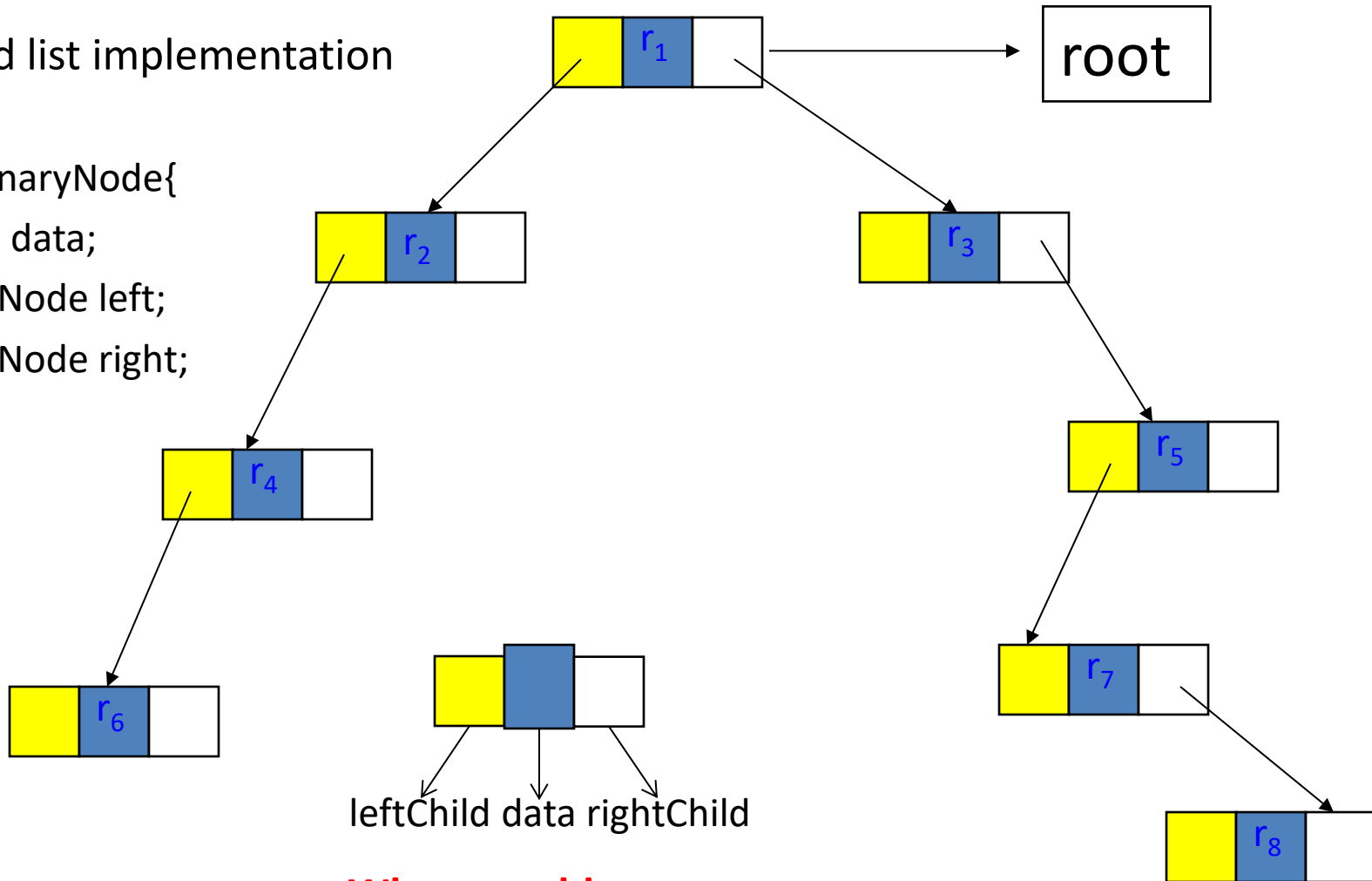


Array size: binary tree with height h , $[h+1, 2^{h+1} - 1]$

Implementation of binary tree contd.

- Linked list implementation

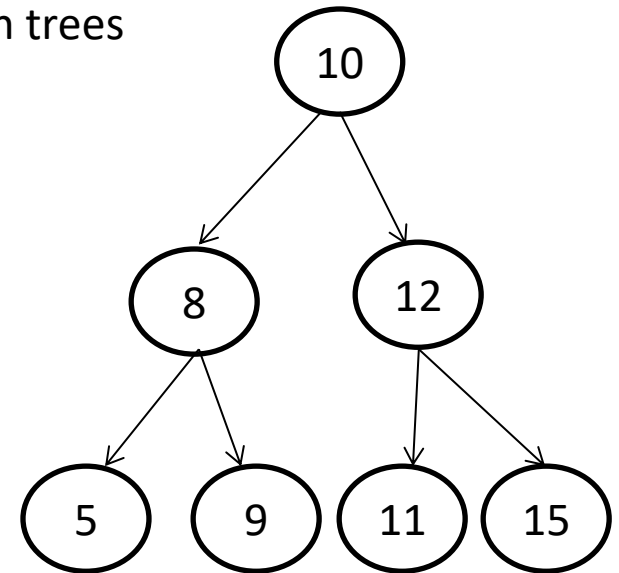
```
Class BinaryNode{  
  Object data;  
  BinaryNode left;  
  BinaryNode right;  
}
```



**When would you use a
linked list???**

Binary search tree

- A binary search tree (BST), is binary tree having the following properties:
 - (i) each node has a (key, value) pair
 - (ii) the left sub-tree of a node r_i contains only nodes with keys less than the key of r_i
 - (iii) the right sub-tree of a node r_i contains only nodes with keys greater than or equal to the key of r_i
 - (iv) both the left and right sub-trees are also binary search trees
- Note that the tree is ordered
- For the purpose of illustration we use integers as keys and assuming their natural order
- Leftmost leaf is always contains the minimum key
- Rightmost leaf always contains the maximum key



Construction

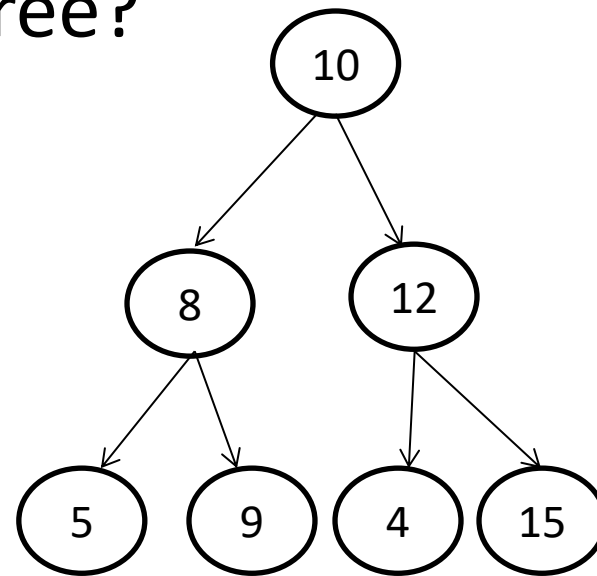
- From a supplied array (eg, 1,5,6,9,11...)– Likely to be systematically unbalanced
- From a random array– Make first element root and inspect sequentially– Likely to be somewhat unbalanced
- Sort the array first– Get the Middle of the array and make it root.– Recursively do same for left half and right half.– Middle of left half becomes the left child of root.– Middle of right half becomes the left child of root.
 - Repeat
- **Computationally expensive! Inflexible**

How to avoid an unbalanced tree

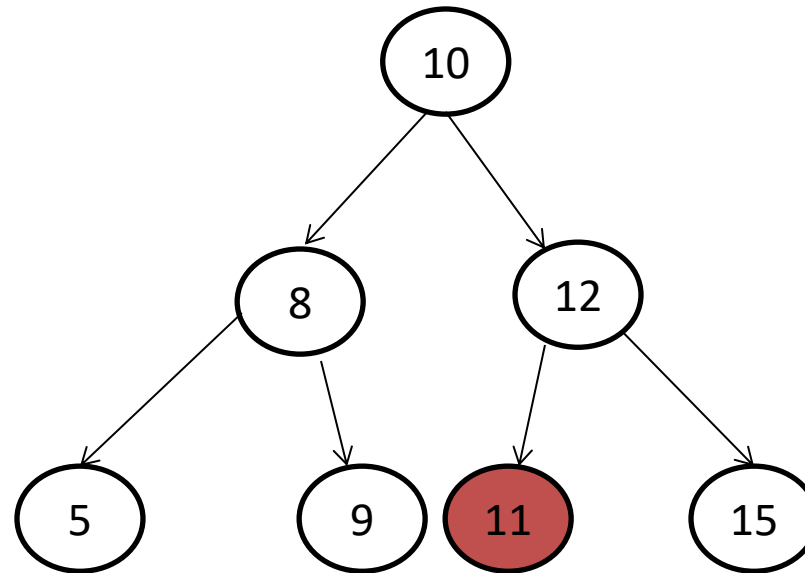
- Randomization
 - If we insert the element into the tree in a random order, the tree generally turns out to be balanced enough. Though often nearer $O(\sqrt{n})$ than $O(\log n)$
- Dynamic Rebalancing.
 - After every insert operation we rebalance the tree so that insert and search are both logarithmic time. (AVL Trees)
- Sort before building ($n \log n$)
- Other (eg. Amortization, splay trees)

Binary search tree

- Is this a binary search tree?

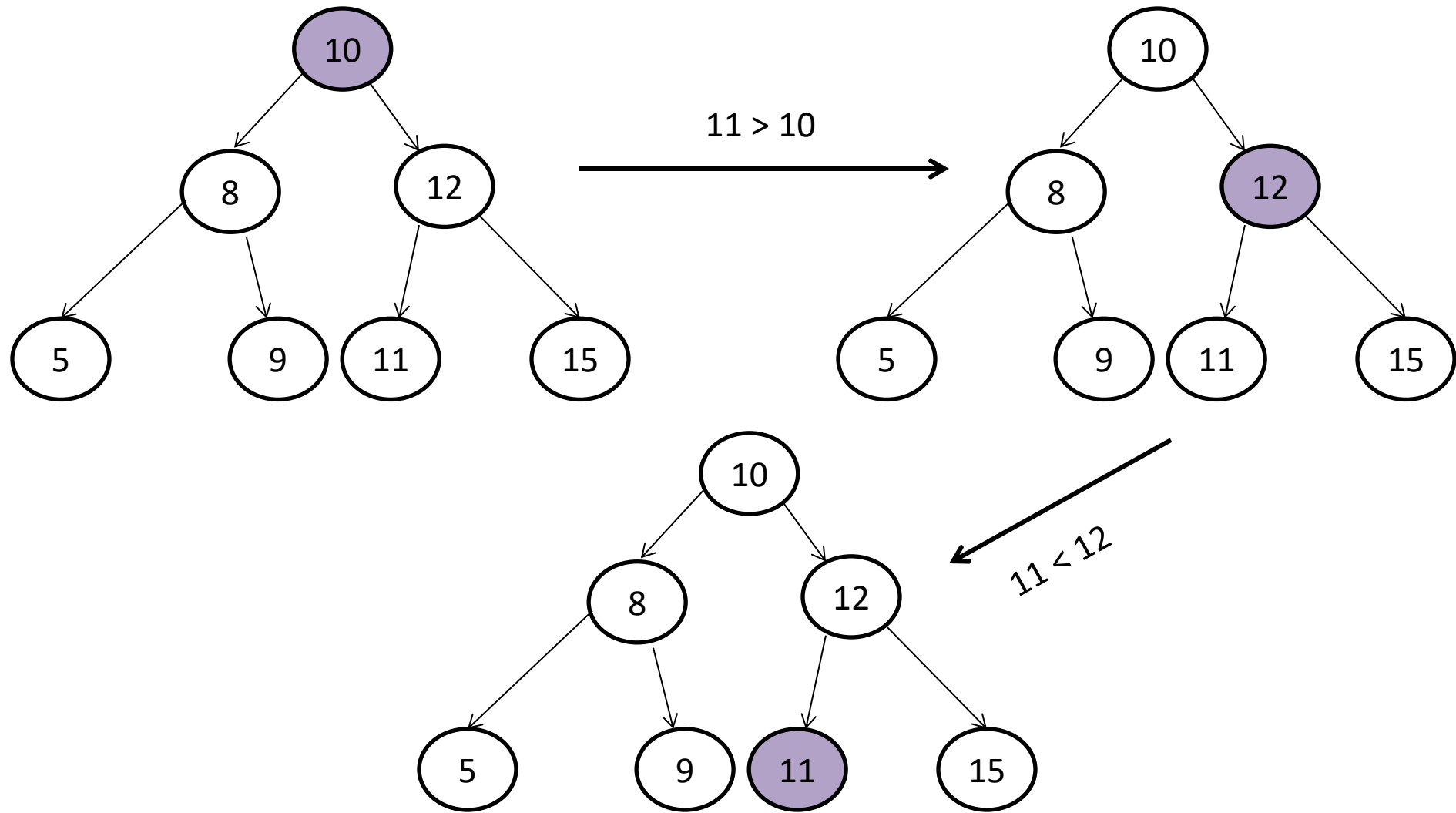


BST Search operation

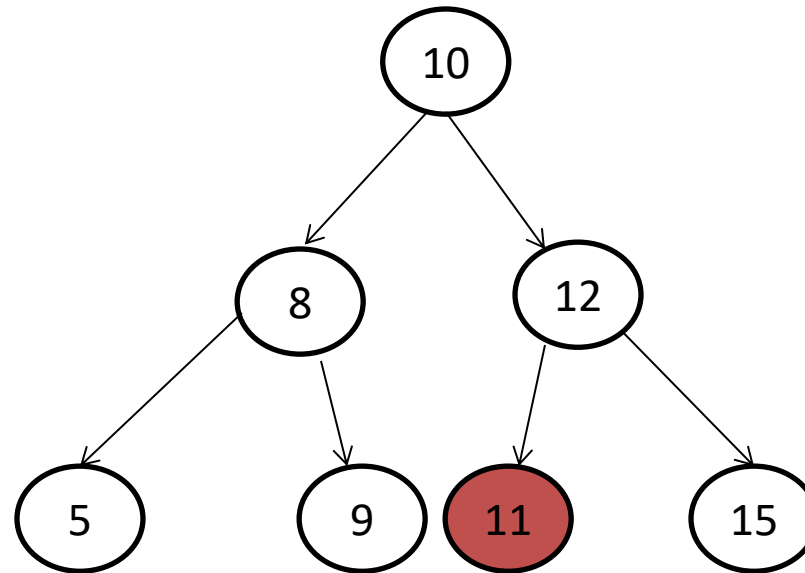


- Search node having key 11

BST Search operation



BST Search operation

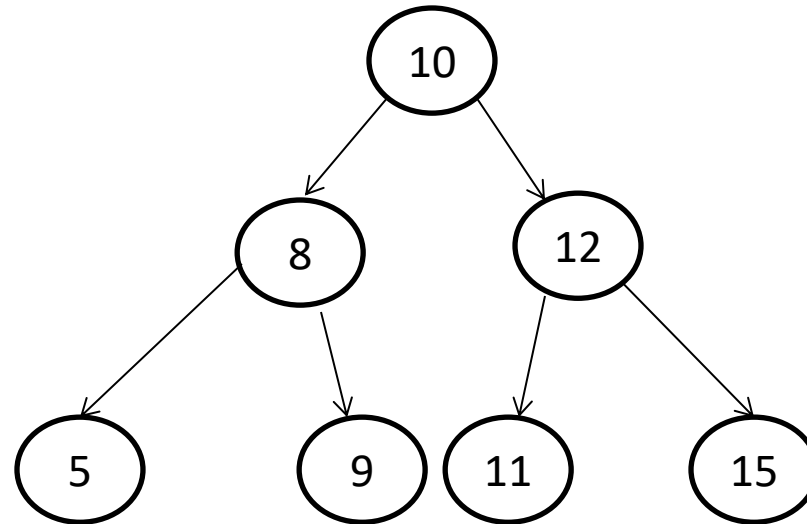


- $O(h)$ in the general, where h is the height of the tree

BST Insertion operation

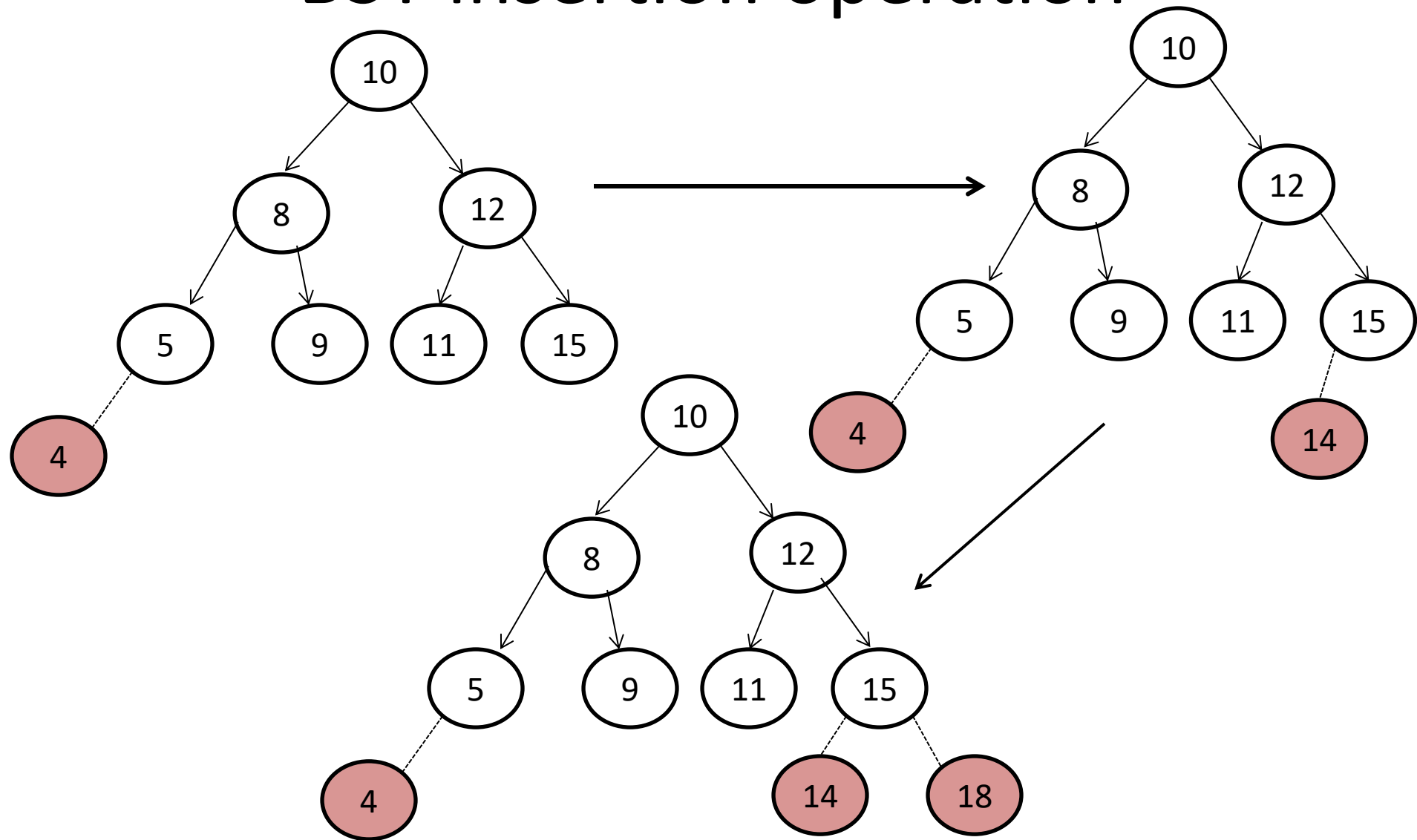
```
if T is empty
    create a root node with the new key
else
    compare key with the root node
    if key == root key
        replace the root with the new value
    else if key > root key
        compare key with the right subtree:
            if subtree is empty create a leaf node
            else add key in right subtree
    else key < root key
        compare key with the left subtree:
            if the subtree is empty create a leaf node
            else add key to the left subtree
```

BST insertion operation

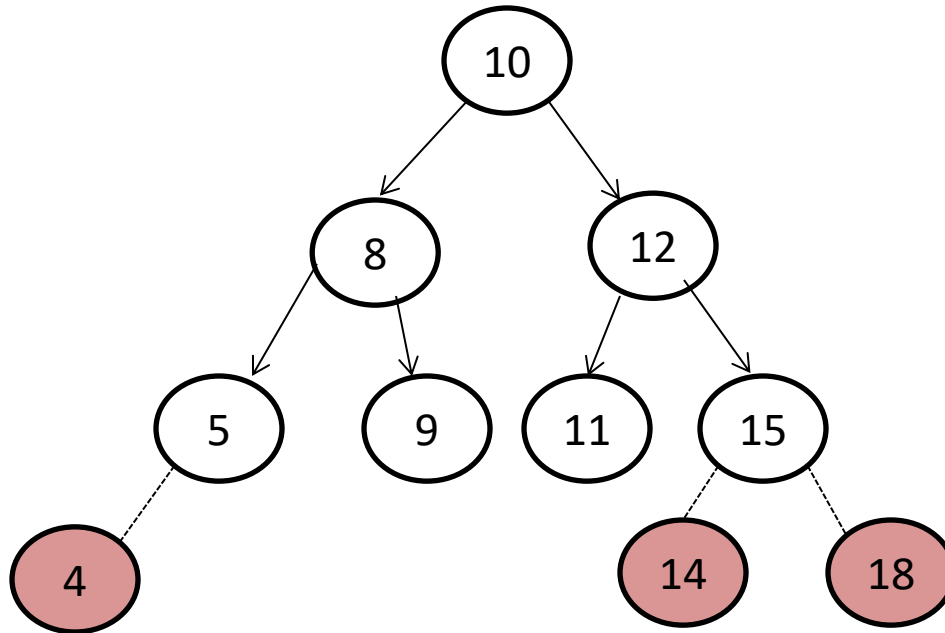


- Insert nodes with keys 4, 14, 18

BST insertion operation



BST Insertion operation



- $O(h)$ in the general, where h is the height of the tree ($O(\log(n))$)

BST Deletion operation

- I if the tree is empty return false
- II Attempt to locate the node containing the target using the binary search algorithm
 - if the target is not found return false
 - else the target is found, so delete its node:
 - Case 1: if the node has 2 empty subtrees
 - replace the link in the parent with null
 - Case 2: if the node has a left and a right subtree
 - replace the node's value with the max value in the left subtree
 - delete the max node in the left subtree
 - Case 3: if the node has no left child
 - link the parent of the node
 - to the right (non-empty) subtree
 - Case 4: if the node has no right child
 - link the parent of the target
 - to the left (non-empty) subtree

BST Deletion operation

- I if the tree is empty return false
- II Attempt to locate the node containing the target using the binary search algorithm
 - if the target is not found return false
 - else the target is found, so delete its node:

**Case 1: if the node has 2 empty subtrees
replace the link in the parent with null**

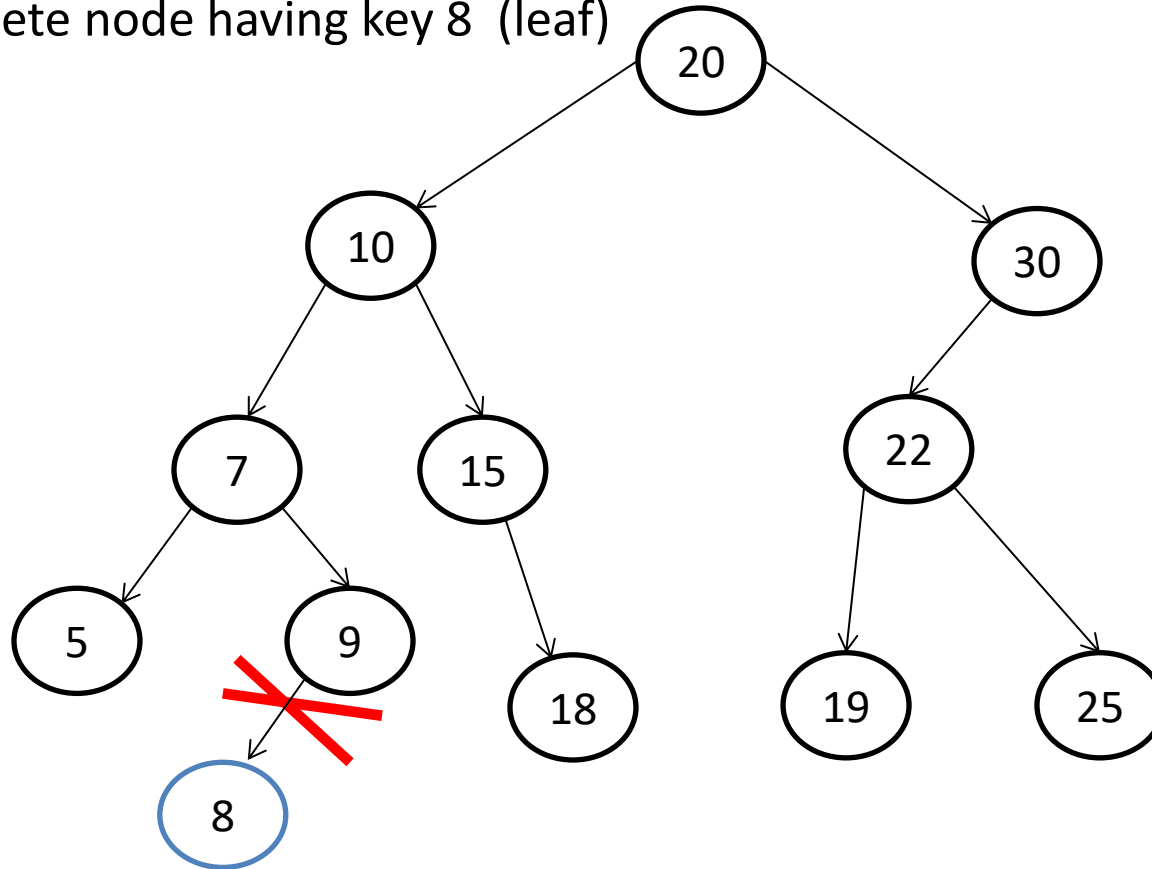
- Case 2: if the node has a left and a right subtree
- replace the node's value with the max value in the left subtree
 - delete the max node in the left subtree

- Case 3: if the node has no left child
- link the parent of the node
 - to the right (non-empty) subtree

- Case 4: if the node has no right child
- link the parent of the target
 - to the left (non-empty) subtree

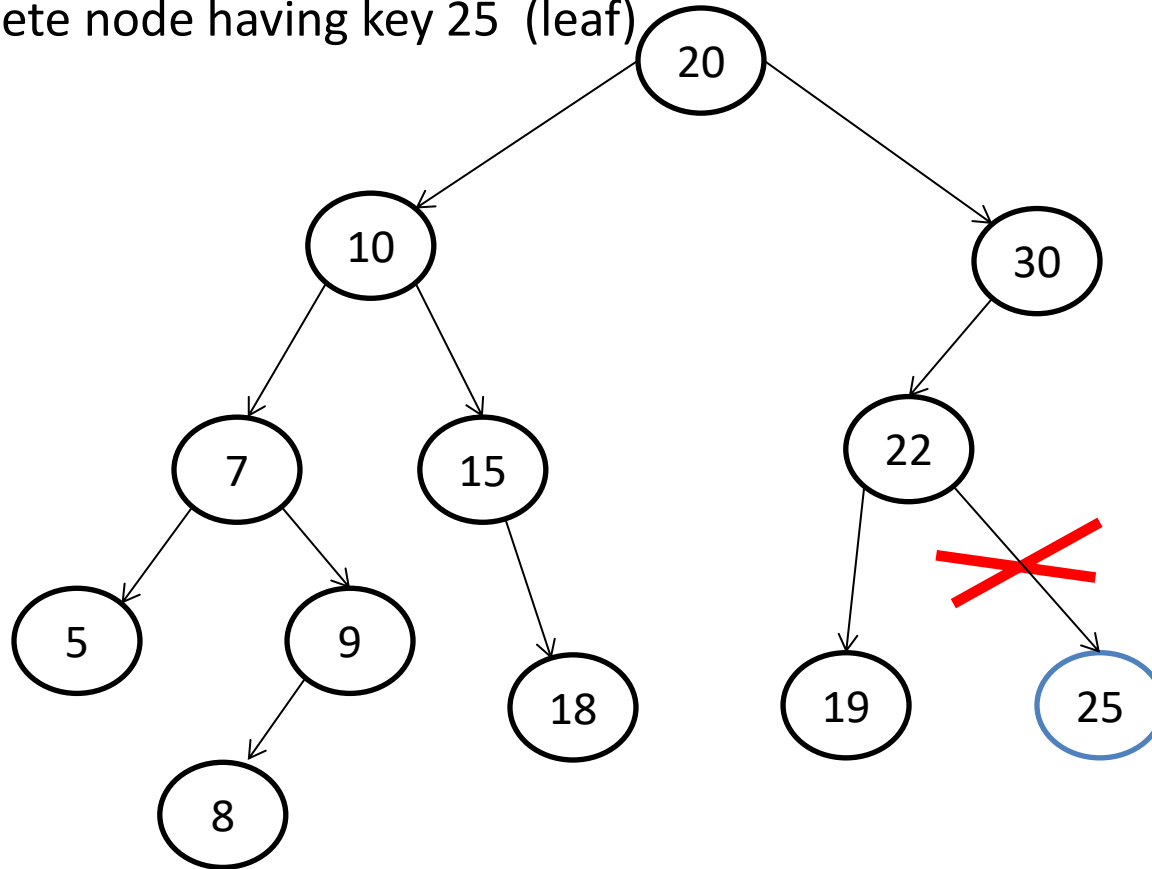
BST Deletion operation

- Delete node having key 8 (leaf)



BST Deletion operation

- Delete node having key 25 (leaf)



BST Deletion operation

```
I  if the tree is empty return false
II Attempt to locate the node containing the target using the binary search
    algorithm
    if the target is not found return false
    else the target is found, so delete its node:
```

```
    Case 1:  if the node has 2 empty subtrees
              replace the link in the parent with null
```

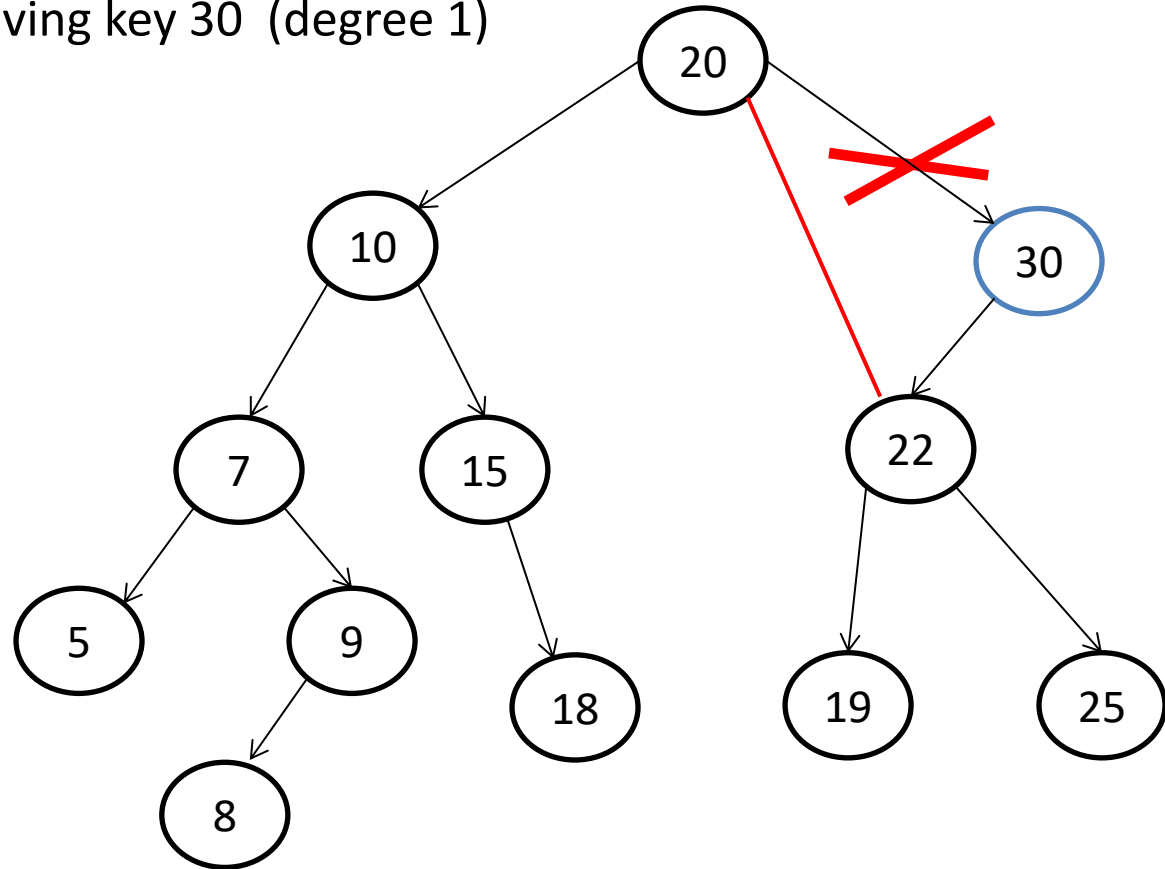
```
    Case 2:  if the node has a left and a right subtree
              - replace the node's value with the max value in the
                left subtree
              - delete the max node in the left subtree
```

```
    Case 3:  if the node has no left child
              - link the parent of the node
              - to the right (non-empty) subtree
```

```
    Case 4:  if the node has no right child
              - link the parent of the target
              - to the left (non-empty) subtree
```

BST Deletion operation

- Delete node having key 30 (degree 1)

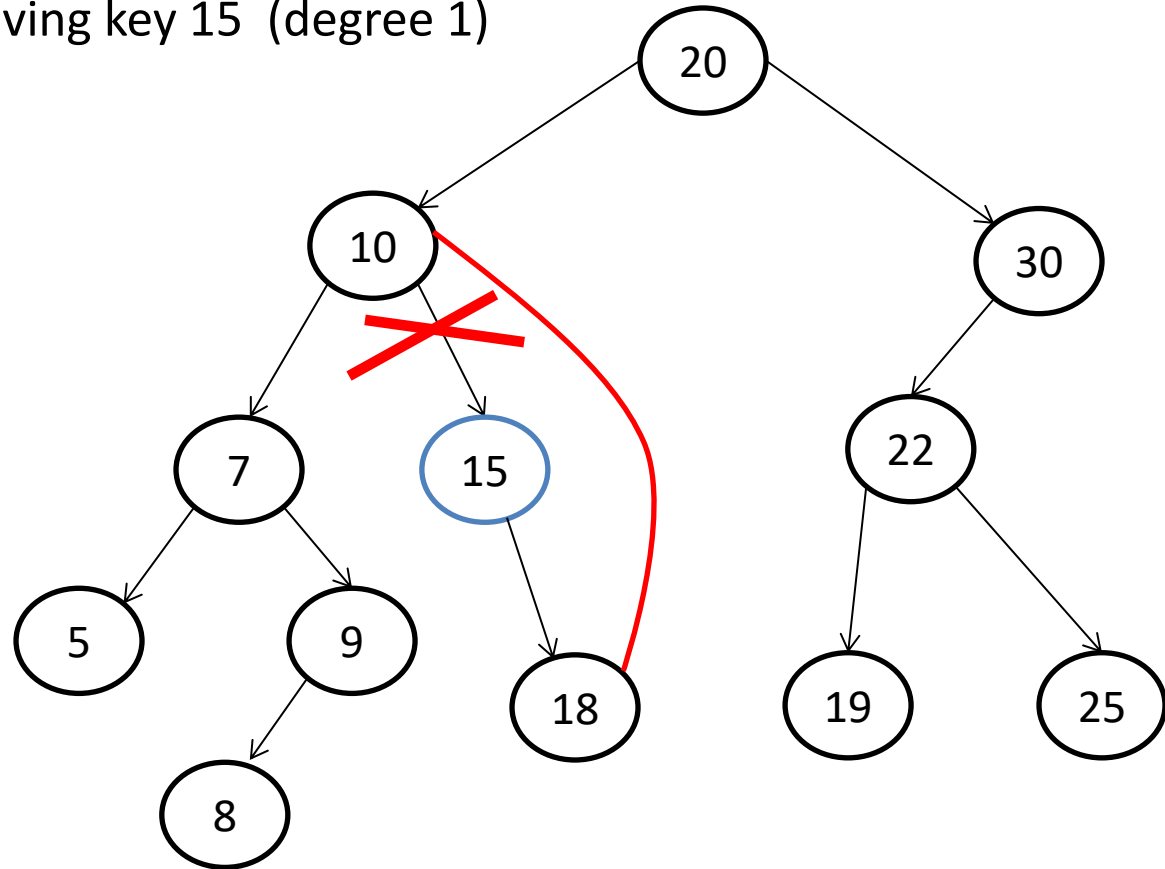


BST Deletion operation

- I if the tree is empty return false
- II Attempt to locate the node containing the target using the binary search algorithm
 - if the target is not found return false
 - else the target is found, so delete its node:
 - Case 1: if the node has 2 empty subtrees
 - replace the link in the parent with null
 - Case 2: if the node has a left and a right subtree
 - replace the node's value with the max value in the left subtree
 - delete the max node in the left subtree
 - Case 3: if the node has no left child**
 - link the parent of the node**
 - to the right (non-empty) subtree**
 - Case 4: if the node has no right child
 - link the parent of the target
 - to the left (non-empty) subtree

BST Deletion operation

- Delete node having key 15 (degree 1)



BST Deletion operation

```
I  if the tree is empty return false
II Attempt to locate the node containing the target using the binary search
    algorithm
    if the target is not found return false
    else the target is found, so delete its node:
```

```
    Case 1:  if the node has 2 empty subtrees
              replace the link in the parent with null
```

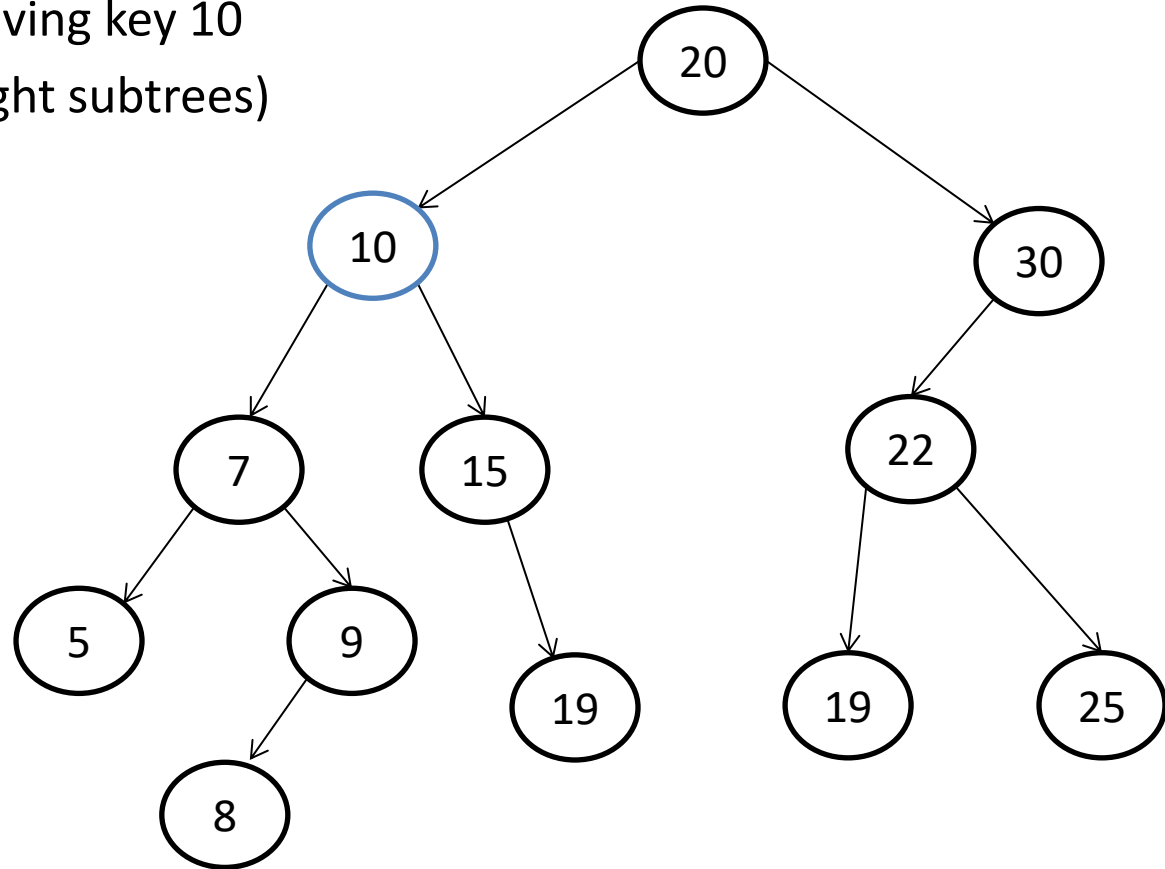
```
    Case 2:  if the node has a left and a right subtree
    - replace the node's value with the max value in the
left subtree
    - delete the max node in the left subtree
```

```
    Case 3:  if the node has no left child
              - link the parent of the node
              - to the right (non-empty) subtree
```

```
    Case 4:  if the node has no right child
              - link the parent of the target
              - to the left (non-empty) subtree
```

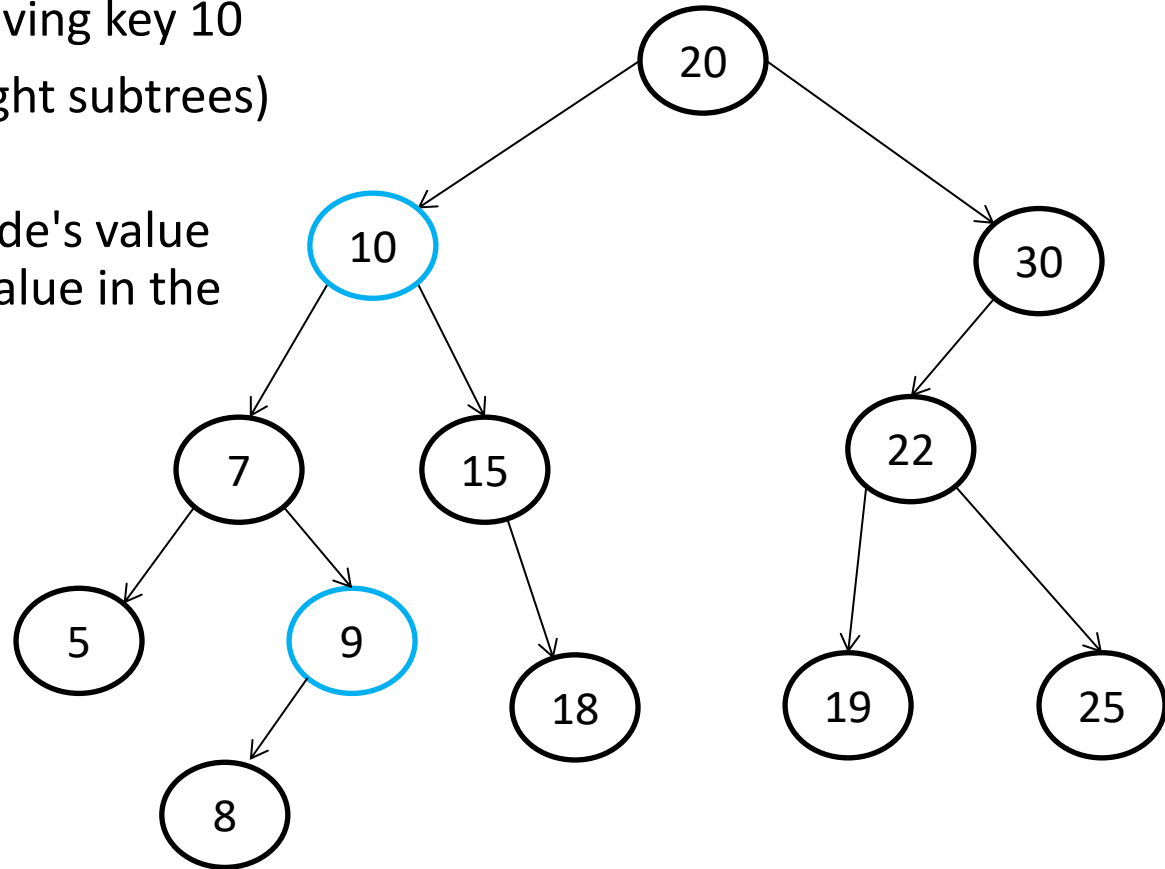
BST Deletion operation

- Delete node having key 10
(has left and right subtrees)



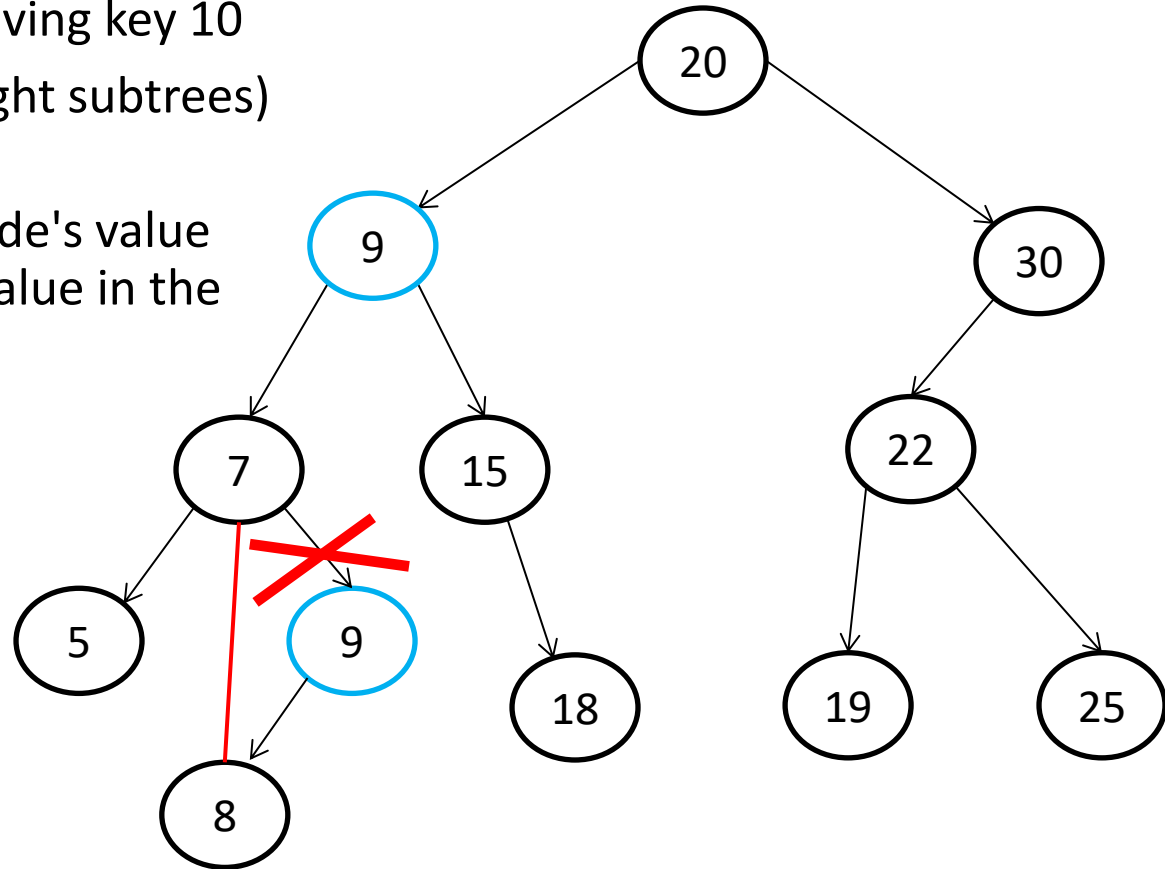
BST Deletion operation

- Delete node having key 10
(has left and right subtrees)
- Replace the node's value
with the max value in the
left subtree



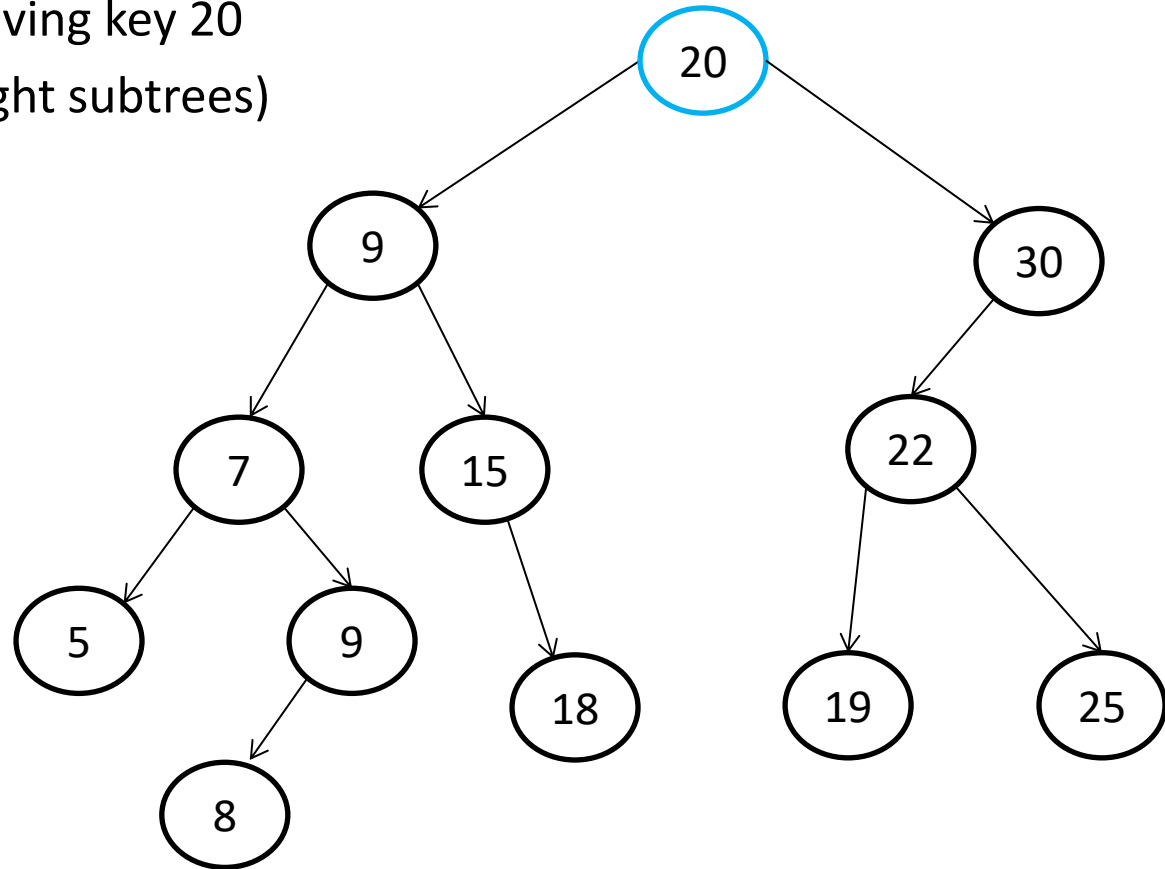
BST Deletion operation

- Delete node having key 10
(has left and right subtrees)
- Replace the node's value with the max value in the left subtree



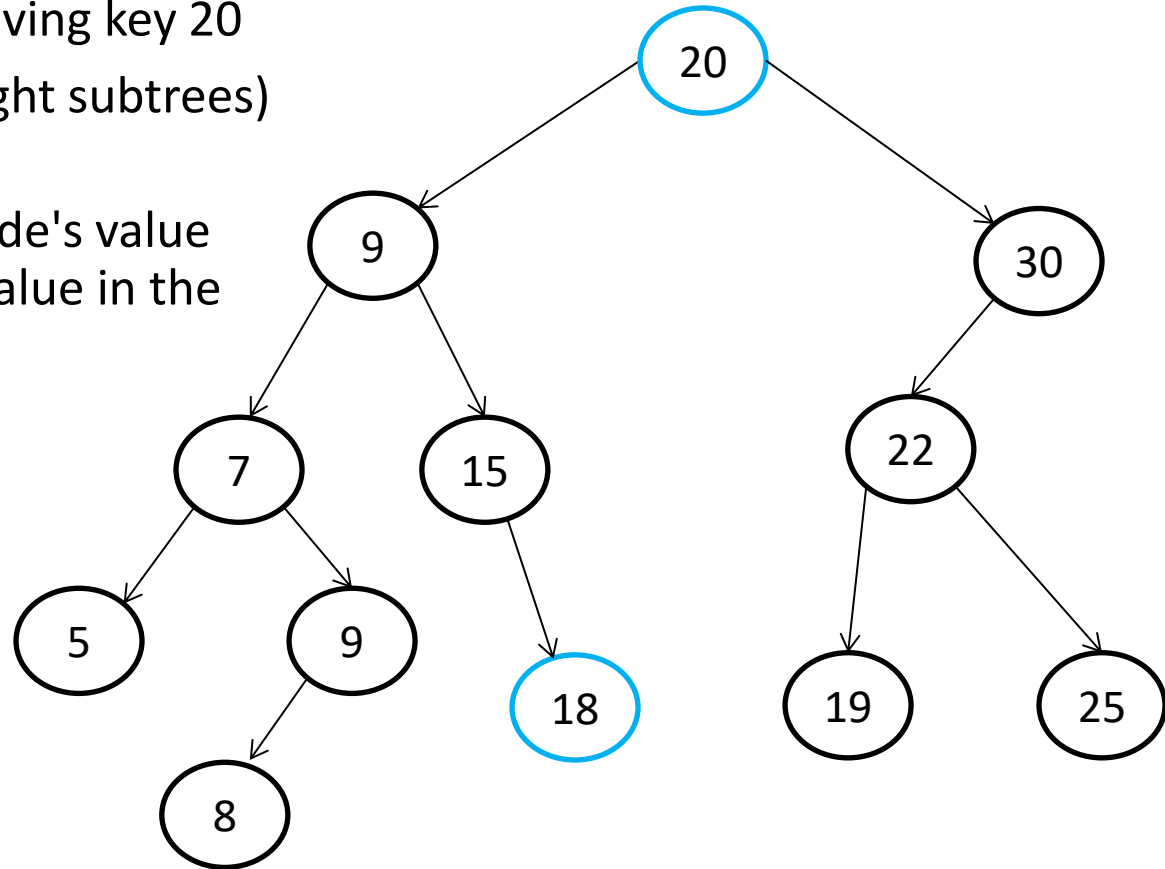
BST Deletion Operation

- Delete node having key 20
(has left and right subtrees)



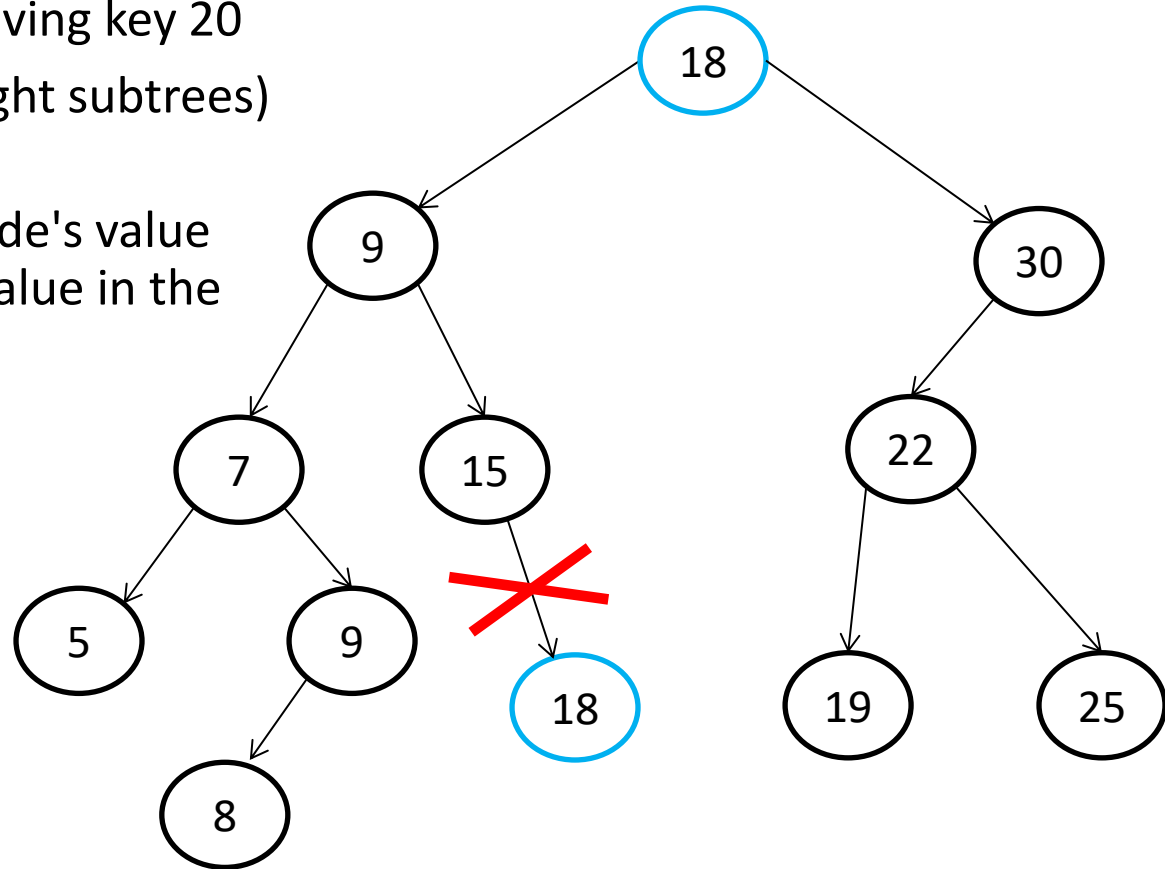
BST Deletion Operation

- Delete node having key 20
(has left and right subtrees)
- Replace the node's value
with the max value in the
left subtree

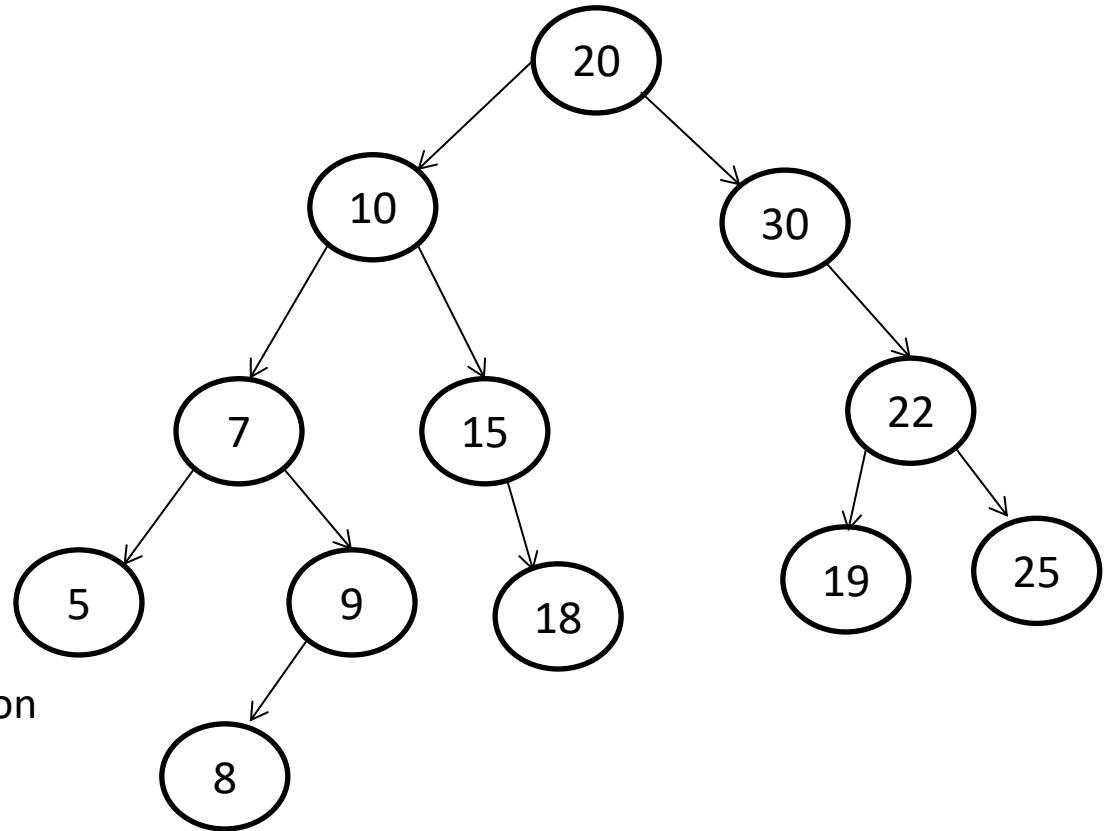


BST Deletion Operation

- Delete node having key 20
(has left and right subtrees)
- Replace the node's value
with the max value in the
left subtree



BST Deletion operation



- $O(\log n)$ complexity for deletion in certain types of the binary search tree (balanced)
- $O(h)$ in the general, where h is the height of the tree

Time complexity for BST operations

	Average	Worst case
Space required	$O(n)$	$O(n)$
Search an item (time)	$O(\log n)$	$O(n)$
Insert an item (time)	$O(\log n)$	$O(n)$
Delete an item (time)	$O(\log n)$	$O(n)$

Prioritised trees

- Place more frequently accessed elements nearer the root
- Easier to do if access requirements known in advance....but rebuilding can be carried out during 'quiet times'
- Examples include dictionaries, criminal records...
- Heaps